

Trabalho Prático #2

NBA

Roberto Rolão de Castro - 107133
Tiago Caridade Gomes - 108307
Sara Figueiredo Almeida - 108796
Joaquim Vertentes Rosa - 109089

Web Semântica - 2024/2025
Mestrado em Engenharia Informática



Conteúdo

1	Introdução ao Tema	4
1.1	Arquitetura da Aplicação e Tecnologias Utilizadas	4
2	Ontologia (RDFS/OWL)	6
2.1	Estrutura Conceptual da Ontologia	6
2.1.1	Classes Principais	6
2.1.2	Propriedades OWL	8
2.1.3	Hierarquia	10
2.2	Axiomas de Restrição	11
3	Regras de Inferência (SPIN)	13
3.1	Conjunto de Inferências Implementadas	13
3.1.1	Inferir Conferência a partir da Divisão	13
3.1.2	Inferir Última Equipa de um Jogador	14
3.1.3	Inferir Início e Fim de Carreira	14
3.1.4	Inferir Idade no Draft	15
3.1.5	Inferir Jogadores “Drafted” no mesmo ano	15
3.2	Adições/Alterações nas <i>Queries</i> SPARQL	15
4	Integração com Dados Externos (Wikidata e DBpedia)	18
4.1	Informações Obtidas da Wikidata	18
4.1.1	Histórico de Treinadores de cada Equipa	18
4.1.2	Informações Detalhadas de Treinadores	19
4.1.3	Prémios e Distinções de Jogadores	19
4.2	Informações Obtidas da DBpedia	20
4.2.1	Detalhes Complementares das Arenas	20
4.3	Sistema de Cache Local	20
5	RDFa e Microformatos	22
5.1	RDFa (RDF Attributes)	22
5.2	Microformatos	24
5.3	Validação	25
6	Funcionalidades adicionais da UI	26
6.1	Inferência na página dos detalhes do jogador	26
6.2	Inferência na página dos detalhes da equipa	26
6.3	Integração com Wikidata — Prémios do Jogador e Treinadores	27
6.4	Integração com DBpedia — Arenas	29
7	Conclusões	30
8	Configuração para executar a aplicação	31
8.1	Executar a aplicação	31
8.2	Ficheiros Importantes	32

Lista de Figuras

2.1	Representação visual da ontologia desenvolvida, destacando as principais classes e relações do domínio NBA.	8
5.1	Resultado da extração RDFa com visualização dos triplos associados ao jogador .	22
5.2	Resultado do parsing RDFa da página HTML de equipas	23
5.3	Resultado do parsing RDFa da página HTML de temporadas	23
5.4	Resultado do parsing RDFa da página HTML das arenas	23
5.5	Resultado do parsing dos Microformatos da página HTML do jogador	24
5.6	Resultado do parsing dos Microformatos da página HTML do treinador	24
5.7	Resultado do parsing dos Microformatos da página HTML da temporada	25
5.8	Resultado do parsing dos Microformatos da página HTML da arena	25
6.1	Novas funcionalidades da página do jogador	26
6.2	Propriedade "conference" na página dos detalhes da equipa	27
6.3	Nova tab dos premios do jogador	27
6.4	Nova página da listagem do treinadores	28
6.5	Nova página dos datalhes do treinador	28
6.6	Nova tab com a lista dos treinadores na página dos detalhes da equipa	29
6.7	Nova tab dos detalhes de contrução de uma arena	29

1. Introdução ao Tema

O presente trabalho prático insere-se no âmbito da unidade curricular de Web Semântica e tem como principal objetivo, estender o trabalho desenvolvido no TP1 e permitir uma compreensão mais expressiva e inteligente dos dados estruturados. O domínio de dados usado volta a centrar-se no domínio da NBA (National Basketball Association), uma das ligas desportivas mais conhecidas a nível mundial, caracterizada por uma estrutura rica em entidades e relações: jogadores, equipas, arenas, temporadas e estatísticas.

Ao longo da execução do trabalho, desenvolvemos uma ontologia para formalizar o domínio do conhecimento. Onde se foi para além dos dados iniciais do RDF, e se introduziu hierarquias para as diferentes classes existentes, restrições de propriedades e axiomas lógicos utilizando RDFS e OWL. Também implementamos mecanismos de inferência baseados em regras *SPIN*, para poderem ser inferidas, para além das já existentes, novas relações entre as classes a partir do dados presentes. Além disso, também usamos as bibliotecas de dados *Wikidata* e *Dbpedia* para complementar e enriquecer ainda mais os dados que já tínhamos do TP1.

A aplicação que resultou deste trabalho permite aos utilizadores explorar e relacionar múltiplas dimensões de dados da NBA através de uma interface web desenvolvida com Django, tal como no primeiro trabalho. O sistema final inclui não só uma base de conhecimento redefinida e semanticamente enriquecida, mas também uma aplicação web de fácil utilização que apresenta e interage com os dados. Suporta consultas inteligentes, raciocínio baseado em regras e publicação semântica através de RDFa e microformatos, demonstrando eficazmente o potencial das tecnologias da Web Semântica na construção de aplicações informativas, interativas e interligadas.

1.1 Arquitetura da Aplicação e Tecnologias Utilizadas

A aplicação foi desenvolvida com base no framework **Django**, seguindo o padrão de arquitetura nativo **MVT (Model-View-Template)**. As *views* tratam os pedidos dos utilizadores e executam queries SPARQL ao triplestore **GraphDB**, enquanto os templates HTML são usados para apresentar os dados de forma dinâmica e interativa. Foi também definido o *model QuizScore*, responsável por armazenar os dados de cada sessão do quiz desenvolvido, permitindo construir um *leaderboard* com as melhores pontuações, visível diretamente na interface do utilizador.

A estrutura do sistema é composta por múltiplos *endpoints* definidos no `urls.py`, que encaixam os pedidos para funções específicas no `views.py`. Estas funções constroem e executam queries SPARQL, processam os resultados e renderizam os templates com os dados.

Tecnologias e Bibliotecas Utilizadas

- **Django** – framework principal para o desenvolvimento do backend e gestão de templates;
- **GraphDB** – base de dados semântica (triplestore) onde os dados RDF da NBA foram carregados e são consultados via SPARQL;
- **SPARQLWrapper** – biblioteca Python usada para construir e executar queries SPARQL a partir das views;
- **Bootstrap** – framework CSS utilizada para o design responsivo e moderno da interface;
- **Chart.js** – biblioteca de visualização de dados usada no dashboard estatístico para gerar gráficos interativos;

- **Leaflet.js** – utilizada para apresentar as arenas da NBA num mapa interativo;
- **vis-network** – usada para representar graficamente a rede de ligações entre jogadores (grafo interativo);
- **AOS (Animate on Scroll)** – adiciona animações visuais suaves ao scroll da página;
- **JavaScript + Fetch API** – utilizado em várias funcionalidades interativas (quiz, filtros, leaderboard, CRUD de jogadores);
- **Docker** – a aplicação foi containerizada para facilitar a sua execução e portabilidade;
- **GraphDB Workbench** – interface gráfica usada para explorar e validar os dados RDF carregados.
- **Protégé** – editor de ontologias usado para modelar e validar o domínio utilizando RDFS e OWL.
- **DBpedia** – usado para enriquecer o conjunto de dados através de queries SPARQL.
- **Wikidata** – utilizado de forma semelhante para integrar informação adicional através de acesso programático.

Esta combinação de tecnologias permitiu criar uma aplicação rica e informativa, baseada em dados semânticos, com uma interface moderna e funcionalidades interativas.

2. Ontologia (RDFS/OWL)

A criação da ontologia constituiu a base conceptual do nosso sistema, permitindo representar formalmente os principais elementos do domínio da NBA, desde jogadores e equipas até arenas, conferências e temporadas. Esta estrutura não só organiza semanticamente os dados, como também facilita inferências automáticas.

No trabalho anterior, os triplos utilizados estavam devidamente identificados com o seu tipo de entidade, por exemplo, `ex:player_201143 rdf:type ex:Player ;`. Já para este projeto, o objetivo era criar uma camada de abstração que separa os dados brutos da sua interpretação semântica, fomentando a reutilização e facilitando a expansão do sistema com novas fontes de conhecimento. Para isto, os dados foram reformulados para que não fosse explícito o seu tipo e aqui entra a ontologia cujo o propósito é permitir que sejam feitas inferências sobre os dados anteriores.

A ontologia foi desenvolvida com recurso às linguagens **RDFS** e **OWL**. O RDFS permitiu a definição das hierarquias básicas de classes e propriedades, enquanto o OWL proporcionou mecanismos mais expressivos, como axiomas de restrições, disjunções entre classes, etc. Esta abordagem híbrida permitiu criar uma base de conhecimento coerente, extensível e semanticamente enriquecida.

Utilizámos o **Protégé** para validar a nossa ontologia, em conjunto com os dados, através das inferências automáticas que lá são feitas com recurso a um reasoner. Esta ferramenta foi muito útil na medida em que é capaz de detetar inconsistências na ontologia e apresentar esses erros. Por fim, antes de avançarmos para a implementação importámos os dados e a ontologia, em separado, no **GraphDB** para testarmos algumas queries e garantirmos a qualidade e completude desta última.

2.1 Estrutura Conceptual da Ontologia

2.1.1 Classes Principais

A ontologia contempla as principais entidades do universo da NBA:

- Person, Player
- Team
- Division
- Conference
- Arena
- State
- Season
- SeasonType
- Position
- Participation

Abaixo apresenta-se a definição destas classes principais, sendo este o início da ontologia:

Listing 2.1: Declaração das principais classes

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix nba: <http://example.org/nba/> .

# CLASSES

nba:Person rdf:type owl:Class .
nba:Team rdf:type owl:Class .
nba:Conference rdf:type owl:Class .
nba:Division rdf:type owl:Class .
nba:Arena rdf:type owl:Class .
nba:Season rdf:type owl:Class .
nba:SeasonType rdf:type owl:Class .
nba:State rdf:type owl:Class .
nba:Position rdf:type owl:Class .
nba:Participation rdf:type owl:Class .

# SUBCLASSES

nba:Player rdf:type owl:Class ;
    rdfs:subClassOf nba:Person .
```

A Figura 2.1 apresenta uma representação gráfica da ontologia, com as classes e suas dependências tal como inferidas pelo Protégé. Assim, este diagrama permite visualizar também a estrutura hierárquica das classes.



Figura 2.1: Representação visual da ontologia desenvolvida, destacando as principais classes e relações do domínio NBA.

2.1.2 Propriedades OWL

Definição das propriedades

A caracterização de cada propriedade das diferentes classes segue toda o mesmo padrão. Isto é, para cada uma são inicialmente definidos: o tipo de propriedade - **rdf:type**, uma anotação do tipo label explicativo - **rdfs:label**, o domínio da propriedade - **rdfs:domain**, a gama de valores que pode assumir **rdfs:range** e, por fim, uma anotação do tipo comentário informativo - **rdfs:comment**.

Exemplo

Listing 2.2: Definição da propriedade *history* da classe *Team*

```

nba:history rdf:type owl:DatatypeProperty ;
  rdfs:label "history" ;
  rdfs:domain nba:Team ;
  rdfs:range xsd:string ;
  rdfs:comment "Information about a team and its history" .
  
```


Tipos das propriedades

Quanto ao tipo, as propriedades definidas podem ser divididas em:

- **Propriedades de tipo de dados** (`owl:DatatypeProperty`), que associam entidades a valores literais (strings, datas, inteiros, etc);
- **Propriedades de objetos** (`owl:ObjectProperty`), que modelam ligações entre instâncias de classes.

Exemplos de propriedades de tipo de dados

Listing 2.3: Fotografia de um jogador - com domain *Player* e gama *xsd:anyURI*

```
nba:playerPhoto rdf:type owl:DatatypeProperty ;
  rdfs:label "playerPhoto" ;
  rdfs:domain nba:Player ;
  rdfs:range xsd:anyURI ;
  rdfs:comment "URL to player's photo" .
```

Listing 2.4: Data de Nascimento de uma pessoa - com domain *Person* e gama *xsd:date*

```
nba:birthdate rdf:type owl:DatatypeProperty ;
  rdfs:label "birthdate" ;
  rdfs:domain nba:Person ;
  rdfs:range xsd:date ;
  rdfs:comment "The birthdate of a person" .
```

Listing 2.5: Capacidade de lugares de uma arena - com domain *Arena* e gama *xsd:nonNegativeInteger*

```
nba:capacity rdf:type owl:DatatypeProperty ;
  rdfs:label "capacity" ;
  rdfs:domain nba:Arena ;
  rdfs:range xsd:nonNegativeInteger ;
  rdfs:comment "The seating capacity of an arena" .
```

Exemplos de propriedades de objetos

Listing 2.6: O *State* onde está localizada uma *Arena*

```
nba:locatedIn rdf:type owl:ObjectProperty ;
  rdfs:label "ArenalocatedIn" ;
  rdfs:domain nba:Arena ;
  rdfs:range nba:State ;
  rdfs:comment "The state of the arena" .
```

Listing 2.7: A *Conference* associada a uma *Division*

```
nba:divisionConference rdf:type owl:ObjectProperty ;
  rdfs:label "conference" ;
  rdfs:domain nba:Division ;
  rdfs:range nba:Conference ;
  rdfs:comment "division conference's name" .
```

Dentro destas propriedades, e como forma de obter uma ontologia mais completa, caracterizámos ainda uma como **propriedade inversa** e sete como **propriedades funcionais**.

Propriedade Inversa

A propriedade inversa serve como forma de definir uma relação bidirecional entre uma *Arena* e a sua *Team* da casa.

Listing 2.8: Propriedade inversa entre `nba:arena` e `nba:homeTeam`

```
nba:arena owl:inverseOf nba:homeTeam .
```

Esta caracterização revela que cada arena tem uma "equipa da casa", tal como cada equipa tem a sua arena associada.

Propriedades Funcionais

As propriedades funcionais são aquelas para as quais existe, no máximo, um valor que podem assumir por cada sujeito. Neste sentido, fazia sentido definir como tal algumas propriedades de um jogador, tais como, `nba:birthdate`, `nba:personName`, `nba:height`, `nba:weight`, `nba:draftYear` e `nba:bornIn` e a propriedade `actualName` de uma equipa, uma vez que esta pode ter vários nomes antigos associados mas apenas um é o atual e mais correto.

Listing 2.9: Propriedades Funcionais

```
nba:birthdate rdf:type owl:FunctionalProperty .
nba:personName rdf:type owl:FunctionalProperty .
nba:height rdf:type owl:FunctionalProperty .
nba:weight rdf:type owl:FunctionalProperty .
nba:draftYear rdf:type owl:FunctionalProperty .
nba:bornIn rdf:type owl:FunctionalProperty .
nba:actualName rdf:type owl:FunctionalProperty .
```

2.1.3 Hierarquia

Para refletir a natureza hierárquica de certos conceitos, foram então estabelecidas relações de herança entre classes. Neste âmbito, e como forma de adicionar profundidade à nossa ontologia, criámos a classe *Person*, inexistente nos dados usados no trabalho passado. Esta é constituída por alguns atributos que eram outrora da classe *Player*, sendo esta última agora uma subclasse da nova classe criada. Neste sentido, um jogador é sempre uma pessoa, mas com propriedades e comportamentos mais especializados:

Listing 2.10: Herança entre *Player* e *Person* e definição das propriedades da nova classe *Person*

```
nba:Player rdf:type owl:Class ;
    rdfs:subClassOf nba:Person .

# PROPERTIES

# Person PROPERTIES

nba:personName rdf:type owl:DatatypeProperty ;
    rdfs:label "personName" ;
    rdfs:domain nba:Person ;
    rdfs:range xsd:string ;
    rdfs:comment "Person's Name" .
```

```
nba:birthdate rdf:type owl:DatatypeProperty ;
  rdfs:label "birthdate" ;
  rdfs:domain nba:Person ;
  rdfs:range xsd:date ;
  rdfs:comment "The birthdate of a person" .

nba:bornIn rdf:type owl:DatatypeProperty ;
  rdfs:label "bornIn" ;
  rdfs:domain nba:Person ;
  rdfs:range xsd:string ;
  rdfs:comment "The place of birth of a person" .
```

Esta estrutura permite então que os jogadores herdem automaticamente as propriedades `nba:birthdate`, `nba:bornIn` e `nba:personName`, evitando redundância e promovendo coerência semântica.

Disjunções Semânticas

Foram também declaradas explicitamente disjunções entre classes que representam conceitos mutuamente exclusivos. Estas disjunções garantem que uma instância não pode pertencer simultaneamente a duas classes semanticamente incompatíveis, assegurando a integridade ontológica:

Listing 2.11: Declaração de disjunções entre classes

```
nba:Player owl:disjointWith nba:Participation .
nba:Team owl:disjointWith nba:Participation .
nba:Team owl:disjointWith nba:Arena .
nba:Team owl:disjointWith nba:Division .
nba:Participation owl:disjointWith nba:Season .
nba:Arena owl:disjointWith nba:State .
```

2.2 Axiomas de Restrição

Restrições de cardinalidade

Finalmente, foram também definidas restrições formais sobre propriedades cruciais do domínio, em particular no que diz respeito às participações. Cada instância da classe `nba:Participation` deve obrigatoriamente ser constituída por exatamente um jogador, uma equipa, uma época e um tipo de época.

Listing 2.12: Cardinalidade obrigatória nas participações

```
nba:Participation rdfs:subClassOf [
  rdf:type owl:Restriction ;
  owl:onProperty nba:player ;
  owl:cardinality "1"^^xsd:nonNegativeInteger
] .

nba:Participation rdfs:subClassOf [
  rdf:type owl:Restriction ;
  owl:onProperty nba:team ;
  owl:cardinality "1"^^xsd:nonNegativeInteger
] .

nba:Participation rdfs:subClassOf [
  rdf:type owl:Restriction ;
  owl:onProperty nba:season ;
  owl:cardinality "1"^^xsd:nonNegativeInteger
] .

nba:Participation rdfs:subClassOf [
```

```
    rdf:type owl:Restriction ;  
    owl:onProperty nba:seasonType ;  
    owl:cardinality "1"^^xsd:nonNegativeInteger  
] .
```

Estas restrições asseguram que todos os registos de participação sejam completos e semanticamente válidos, espelhando corretamente a estrutura da informação no domínio da NBA.

3. Regras de Inferência (SPIN)

Para além da inferência ontológica nativa providenciada pelo OWL (como herança, disjunções ou *domains* e *ranges*), este sistema recorreu a um mecanismo adicional de inferência baseado em regras SPIN (*SPARQL Inferencing Notation*), com o objetivo de enriquecer automaticamente os dados semânticos com base em padrões observados nos triplos RDF.

As regras foram definidas diretamente no módulo Python `spin_rules.py`, implementadas como funções que executam queries SPARQL INSERT. As regras que inferem propriedades que não influenciam a interface implementada anteriormente são executadas automaticamente no startup do projeto Django, enquanto as regras que acrescentam novas informações não disponíveis na UI anterior são executadas através de um botão na interface, seguidas imediatamente de queries para apresentar as novas propriedades adicionadas. Cada regra corresponde a um padrão lógico identificado nos dados, que permite inferir novos triplos de forma programática através de operações de inserção condicionais.

Estas regras de inferência permitiram estender as capacidades de raciocínio para além do que é suportado exclusivamente por OWL, permitindo assim uma análise mais rica e completa da informação contida no grafo de conhecimento.

3.1 Conjunto de Inferências Implementadas

Foram implementadas cinco regras principais de inferência, cada uma com um propósito distinto:

3.1.1 Inferir Conferência a partir da Divisão

Se uma equipa pertence a uma divisão, e essa divisão pertence a uma conferência, então a equipa também pertence à conferência.

Listing 3.1: Inferir conferência com base na divisão

```
PREFIX nba: <http://example.org/nba/>
INSERT {
    ?team nba:conference ?conference .
}
WHERE {
    ?team nba:division ?division .
    ?division nba:divisionConference ?conference .
    FILTER NOT EXISTS { ?team nba:conference ?conference }
}
```

3.1.2 Inferir Última Equipa de um Jogador

Inferir a última equipa por onde passou cada jogador, com base na época mais recente onde participou numa temporada regular.

Listing 3.2: Inferir última equipa com base na época mais recente

```
PREFIX nba: <http://example.org/nba/>
INSERT {
  ?player nba:lastTeam ?team .
}
WHERE {
  ?participation nba:player ?player .
  ?participation nba:team ?team .
  ?participation nba:season ?season .
  ?participation nba:seasonType nba:seasonType_1 .
  {
    SELECT ?player (MAX(?seasonLabel) AS ?maxSeason) WHERE {
      ?participation nba:player ?player .
      ?participation nba:season ?season .
      ?season nba:seasonLabel ?seasonLabel .
      ?participation nba:seasonType nba:seasonType_1 .
    } GROUP BY ?player
  }
  ?season nba:seasonLabel ?maxSeason .
  FILTER NOT EXISTS { ?player nba:lastTeam ?team }
}
```

3.1.3 Inferir Início e Fim de Carreira

Calcular automaticamente os anos de início e fim de carreira de um jogador com base na época mais antiga e mais recente em que participou.

Listing 3.3: Inferir carreira com base nas épocas registadas

```
PREFIX nba: <http://example.org/nba/>
CONSTRUCT {
  ?player nba:careerStart ?firstSeason .
  ?player nba:careerEnd ?lastSeason .
}
WHERE {
  {
    SELECT ?player (MIN(?seasonLabel) AS ?firstSeason) (MAX(?seasonLabel)
    AS ?lastSeason) WHERE {
      ?participation nba:player ?player .
      ?participation nba:season ?season .
      ?season nba:seasonLabel ?seasonLabel .
      ?participation nba:seasonType nba:seasonType_1 .
    } GROUP BY ?player
  }
  FILTER NOT EXISTS {
    ?player nba:careerStart ?firstSeason .
    ?player nba:careerEnd ?lastSeason .
  }
}
```

3.1.4 Inferir Idade no Draft

Calcular a idade aproximada no momento do draft com base no ano de nascimento e no ano do draft.

Listing 3.4: Inferir idade no momento do draft

```
PREFIX nba: <http://example.org/nba/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

CONSTRUCT {
  ?player nba:draftAge ?age .
}
WHERE {
  ?player nba:birthdate ?birthdate .
  ?player nba:draftYear ?draftYearStr .
  BIND(xsd:integer(?draftYearStr) AS ?draftYearInt)
  BIND(?draftYearInt - YEAR(xsd:date(?birthdate)) AS ?age)
}
```

3.1.5 Inferir Jogadores “Drafted” no mesmo ano

Propriedade inferida que associa jogadores que foram “drafted” no mesmo ano que um determinado jogador.

Listing 3.5: Inferir colegas de draft pelo mesmo ano

```
PREFIX nba: <http://example.org/nba/>
CONSTRUCT {
  ?player1 nba:draftedInSameYearAs ?player2 .
}
WHERE {
  ?player1 nba:draftYear ?year .
  ?player2 nba:draftYear ?year .
  FILTER(?player1 != ?player2)
  FILTER NOT EXISTS { ?player1 nba:draftedInSameYearAs ?player2 }
}
```

3.2 Adições/Alterações nas *Queries* SPARQL

Com a adição destas novas inferências, duas *queries* já existentes tiveram de ser alteradas e três novas *queries* foram adicionadas.

A primeira *query* que foi alterada com a inserção destas novas propriedades inferidas, foi a *query* que retorna a informação a ser apresentada na página de uma equipa. Anteriormente a propriedade "conference" era buscada a partir da classe "Division" de uma equipa, e agora está a ser usada diretamente por "Team":

Listing 3.6: Query da informação da equipa usando a propriedade "conference"

```
PREFIX nba: <http://example.org/nba/>

SELECT ?name ?actualName ?acronym ?logo ?city ?statename ?conference
       ?conferencename ?divison ?divisionname ?arena ?arenaname WHERE {
  ?team a nba:Team ;
        nba:actualName ?actualName ;
  OPTIONAL { ?team nba:teamName ?name . }
  ...
  OPTIONAL { ?team nba:conference ?conference . }
  OPTIONAL { ?conference nba:conferenceName ?conferencename . }
  ...
  FILTER(STR(?team) = "{team_uri}")
}
```

}

A segunda *query* alterada foi a que retorna as informações para ser apresentadas na página de um determinado jogador. Anteriormente, a última equipa do jogador era obtida a partir de uma *query* mais complexa, que foi completamente substituída pela utilização da nova propriedade "lastTeam":

Listing 3.7: Query da informação do jogador usando a propriedade "lastTeam"

```
PREFIX nba: <http://example.org/nba/>

SELECT ?name ?birthdate ?bornIn ?draftYear ?position ?height ?weight ?school
       ?photo
       ?teamId ?teamName ?teamLogo WHERE {
  ?player a nba:Player ;
          nba:personName ?name .
  FILTER(STR(?player) = "{jogador_uri}")

  ...

  # Use the inferred lastTeam property
  OPTIONAL {
    ?player nba:lastTeam ?teamId .
    ?teamId nba:actualName ?teamName ;
            nba:logo ?teamLogo .
  }
}
```

A primeira nova *query* permite obter os jogadores que foram seleccionados no mesmo ano de draft que um determinado jogador, utilizando a propriedade "draftedInSameYearAs":

Listing 3.8: Query para obter com mesmo ano de draft

```
PREFIX nba: <http://example.org/nba/>
SELECT ?playerName ?draftYear ?draftmate ?draftmateName ?draftmatePhoto
       ?position WHERE {
  <PLAYER_URI> nba:personName ?playerName ;
               nba:draftYear ?draftYear ;
               nba:draftedInSameYearAs ?draftmate .
  ?draftmate nba:personName ?draftmateName .
  OPTIONAL { ?draftmate nba:playerPhoto ?draftmatePhoto . }
  OPTIONAL {
    ?draftmate nba:position ?posObj .
    ?posObj nba:positionName ?position .
  }
}
ORDER BY ?draftmateName
```

A segunda nova *query* consulta o período de carreira de um jogador através das propriedades "careerStart" e "careerEnd":

Listing 3.9: Query para obter período de carreira

```
PREFIX nba: <http://example.org/nba/>
SELECT ?careerStart ?careerEnd WHERE {
  <PLAYER_URI> nba:careerStart ?careerStart ;
               nba:careerEnd ?careerEnd .
}
```


A terceira nova *query* obtém a idade de um jogador no momento do draft através da propriedade "draftAge":

Listing 3.10: Query para obter idade do jogador no draft

```
PREFIX nba: <http://example.org/nba/>

SELECT ?age WHERE {
  ?player a nba:Player ;
          nba:draftAge ?age .
  FILTER(STR(?player) = "http://example.org/nba/player_ID")
}
```

4. Integração com Dados Externos (Wikidata e DBpedia)

A interligação com dados externos permite aumentar significativamente a riqueza semântica da nossa base de dados em GraphDB já existente, através da incorporação de informação atualizada e verificada, ao mesmo tempo que demonstra a interoperabilidade entre ontologias distintas e possibilita a validação e cruzamento de informação com fontes abertas de dados como a Wikidata e DBpedia.

Para complementar os dados NBA existentes no sistema, foi desenvolvido um módulo de integração (`data_complementation.py`) que permite aceder programaticamente a informações adicionais através de consultas SPARQL a endpoints externos. Este módulo implementa uma arquitetura de cache local para otimizar o desempenho e reduzir a latência das consultas, armazenando temporariamente os resultados obtidos das fontes externas. Nas secções seguintes estão apresentadas as *queries* que foram implementadas.

4.1 Informações Obtidas da Wikidata

A Wikidata foi utilizada como fonte principal para obter informações biográficas e históricas sobre treinadores e jogadores NBA. As consultas implementadas permitem obter:

4.1.1 Histórico de Treinadores de cada Equipa

Esta consulta permite identificar todos os treinadores que dirigiram uma determinada equipa NBA ao longo da história, incluindo os períodos de exercício das funções.

Listing 4.1: Query para histórico de treinadores de cada equipa

```
PREFIX wd: <http://www.wikidata.org/entity/>
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?coach ?coachLabel ?teamLabel ?image ?start_date ?end_date
WHERE {
  ?team wdt:P31 wd:Q13393265 ;      # instance of basketball team
        wdt:P118 wd:Q155223 ;      # league: National Basketball Association
        rdfs:label ?teamLabel .

  ?coach p:P6087 ?statement .      # statement where coach coached a team
        ?statement ps:P6087 ?team .      # the actual team coached
        OPTIONAL { ?statement pq:P580 ?start_date . } # start time
        OPTIONAL { ?statement pq:P582 ?end_date . }   # end time

  ?coach wdt:P106 wd:Q5137571 ;     # occupation: basketball coach
        rdfs:label ?coachLabel .
        OPTIONAL { ?coach wdt:P18 ?image . }          # image

  FILTER(LANG(?teamLabel) = "en")
  FILTER(LANG(?coachLabel) = "en")
  FILTER(CONTAINS(LCASE(?teamLabel), LCASE("{clean_name}")))

  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
```

4.1.2 Informações Detalhadas de Treinadores

Para cada treinador identificado, é possível obter informações biográficas completas, incluindo formação académica, nacionalidade e histórico profissional.

Listing 4.2: Query para informações detalhadas de treinadores

```
PREFIX wd: <http://www.wikidata.org/entity/>
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX p: <http://www.wikidata.org/prop/>
PREFIX ps: <http://www.wikidata.org/prop/statement/>
PREFIX pq: <http://www.wikidata.org/prop/qualifier/>

SELECT ?coachLabel ?dob ?pobLabel ?genderLabel ?citizenshipLabel ?languageLabel
       ?positionLabel ?image ?givenNameLabel ?familyNameLabel
       ?schoolLabel ?trainerLabel ?teamLabel ?start ?end
WHERE {
  wd:{coach_entity_id} rdfs:label ?coachLabel .
  FILTER(LANG(?coachLabel) = "en")

  OPTIONAL { wd:{coach_entity_id} wdt:P569 ?dob . }
  OPTIONAL { wd:{coach_entity_id} wdt:P19 ?pob . }
  OPTIONAL { wd:{coach_entity_id} wdt:P21 ?gender . }
  OPTIONAL { wd:{coach_entity_id} wdt:P27 ?citizenship . }
  OPTIONAL { wd:{coach_entity_id} wdt:P103 ?language . }
  OPTIONAL { wd:{coach_entity_id} wdt:P413 ?position . }
  OPTIONAL { wd:{coach_entity_id} wdt:P18 ?image . }
  OPTIONAL { wd:{coach_entity_id} wdt:P735 ?givenName . }
  OPTIONAL { wd:{coach_entity_id} wdt:P734 ?familyName . }
  OPTIONAL { wd:{coach_entity_id} wdt:P69 ?school . }
  OPTIONAL { wd:{coach_entity_id} wdt:P1066 ?trainer . }

  OPTIONAL {
    wd:{coach_entity_id} p:P6087 ?coachStatement .
    ?coachStatement ps:P6087 ?team ;
                    pq:P580 ?start ;
                    pq:P582 ?end .
    ?team rdfs:label ?teamLabel .
    FILTER(LANG(?teamLabel) = "en")
  }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}
```

4.1.3 Prêmios e Distinções de Jogadores

Esta consulta permite identificar todos os prêmios e reconhecimentos recebidos por jogadores NBA registados na Wikidata.

Listing 4.3: Query para prêmios de jogadores

```
PREFIX wd: <http://www.wikidata.org/entity/>
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?player ?playerLabel ?award ?awardLabel WHERE {
  ?player wdt:P106 wd:Q3665646 ;           # occupation: basketball player
          wdt:P118 wd:Q155223 ;           # league: NBA
          rdfs:label ?playerLabel ;
          wdt:P166 ?award .               # award received

  ?award rdfs:label ?awardLabel .
}
```

```

FILTER(LANG(?playerLabel) = "en")
FILTER(LANG(?awardLabel) = "en")
FILTER(CONTAINS(LCASE(?playerLabel), LCASE("{clean_player_name}")))

SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}

```

4.2 Informações Obtidas da DBpedia

A DBpedia foi utilizada para complementar os dados sobre as arenas NBA, que careciam de informação.

4.2.1 Detalhes Complementares das Arenas

Esta consulta permite obter algumas informações sobre a construção e arquitetura das arenas NBA, incluindo custos de construção, arquitetos responsáveis e datas relevantes.

Listing 4.4: Query para detalhes complementares de pavilhões

```

PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX foaf: <http://foaf.org/0.1/>
PREFIX dct: <http://purl.org/dc/terms/>
PREFIX dbp: <http://dbpedia.org/property/>

SELECT DISTINCT ?arena ?label ?architect ?architectName ?architectThumbnail
?constructionCost ?openingDate ?buildingStartDate WHERE {
  ?arena a dbo:Stadium ;
         rdfs:label ?label .

  OPTIONAL {
    ?arena dbp:architect ?architect .
    OPTIONAL { ?architect rdfs:label ?architectName .
      FILTER(LANG(?architectName) = "en") }
    OPTIONAL { ?architect dbo:thumbnail ?architectThumbnail . }
  }
  OPTIONAL { ?arena dbp:cost ?constructionCost . }
  OPTIONAL { ?arena dbp:cost ?constructionCost . }
  OPTIONAL { ?arena dbp:opened ?openingDate . }
  OPTIONAL { ?arena dbp:buildingStartDate ?buildingStartDate . }

  FILTER(LANG(?label) = "en")
  FILTER(CONTAINS(LCASE(?label), LCASE("{clean_name}")))
}

```

4.3 Sistema de Cache Local

Para otimizar o desempenho e reduzir a dependência de conectividade externa, foi implementado um sistema de cache que armazena temporariamente os resultados das consultas a fontes externas. O sistema utiliza a infraestrutura de cache em memória local do Django, guardando estas informações externas durante um período de 12 horas.

Nesta implementação, o sistema primeiro verifica se os dados solicitados já existem no cache local. Em caso de *cache hit*, os dados são retornados imediatamente, evitando a consulta aos endpoints externos. Quando ocorre um *cache miss*, a consulta SPARQL é executada normalmente e os resultados são armazenados no cache para consultas futuras.

Listing 4.5: Exemplo de utilização do sistema de cache

```

def get_team_coaches_from_wikidata(self, team_name: str):

```

```

cache_key = self._generate_cache_key('team_coaches', team_name)
return self._get_from_cache_or_fetch(
    cache_key,
    self._fetch_team_coaches_from_wikidata,
    self.cache_timeouts['team_coaches'],
    team_name
)

```

Listing 4.6: Método de gestão da cache

```

def _get_from_cache_or_fetch(self, cache_key: str, fetch_function, timeout:
    int, *args, **kwargs):
    # Try to get from cache first
    cached_data = cache.get(cache_key)
    if cached_data is not None:
        return cached_data

    # If not in cache, fetch the data
    fresh_data = fetch_function(*args, **kwargs)

    # Store in cache for future use
    if fresh_data: # Only cache non-empty results
        cache.set(cache_key, fresh_data, timeout)

    return fresh_data

```

O sistema de cache melhora significativamente a experiência do utilizador ao reduzir a latência das consultas repetidas e também pode funcionar como mecanismo de resiliência em caso de falhas temporárias de conectividade com os serviços externos.

5. RDFa e Microformatos

Para garantir que a riqueza semântica dos dados não seja posta em causa e seja perceptível tanto aos utilizadores humanos como para máquinas, a aplicação desenvolvida incorpora anotações semânticas diretamente no *HTML* das suas páginas. Isto só é possível graças a utilização de *RDFa* e *Microformatos*, duas abordagens complementares para incorporar dados semânticos estruturados em conteúdos de páginas *HTML*.

5.1 RDFa (RDF Attributes)

O RDFa foi usado para incorporar triplos RDF nas visualizações da aplicação, permitindo um mapeamento claro entre os conteúdos da página *HTML* e o modelo semântico. Isto vai permitir que agentes como *crawlers* ou parsers semânticos extraiam o significado diretamente da página sem a necessidade de um ficheiro RDF separado.

Na nossa aplicação o *RDFa* foi implementado em:

- `/jogador/id/ - templates/player.html`: Na primeira foto pode se ver as propriedades do um jogador, já na segunda também se pode ver essas informações mas também todos os jogadores que alguma vez jogaram com o jogador em questão:

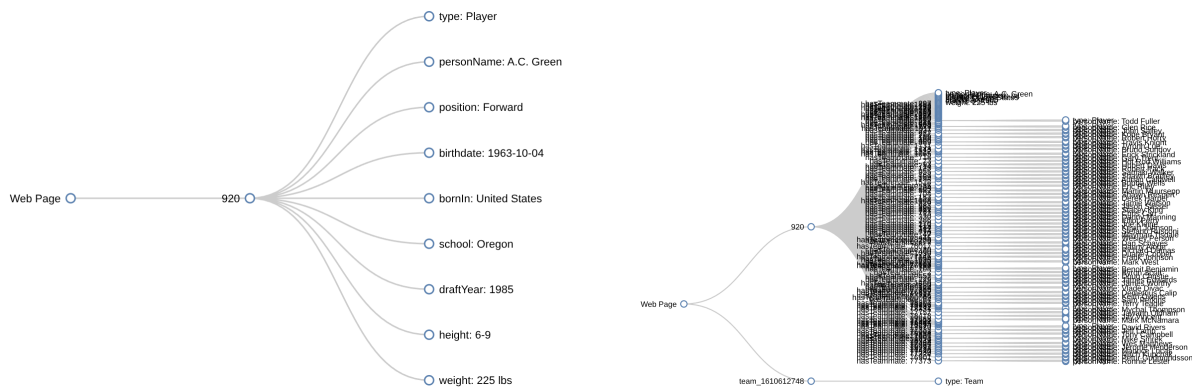


Figura 5.1: Resultado da extração RDFa com visualização dos triplos associados ao jogador

- `/teams/ - templates/teams.html:`

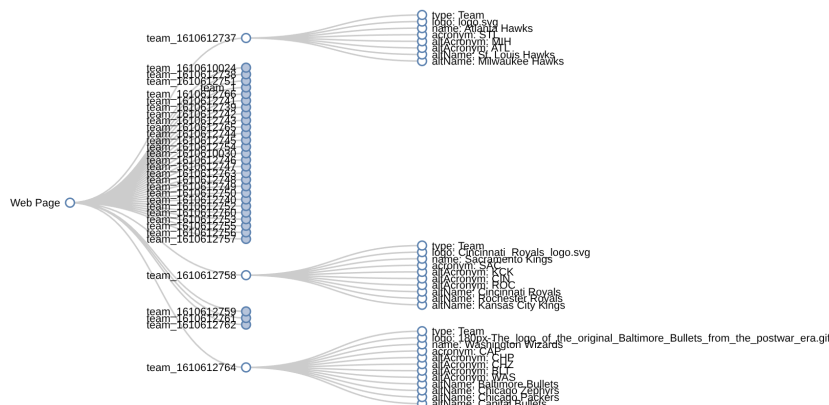


Figura 5.2: Resultado do parsing RDFa da página HTML de equipas

- `/seasons/ - templates/temporadas.html:`

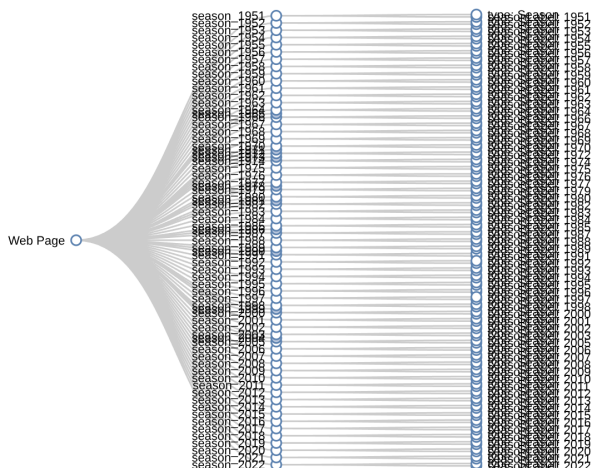


Figura 5.3: Resultado do parsing RDFa da página HTML de temporadas

- `/arenas/ - templates/arenas.html:`

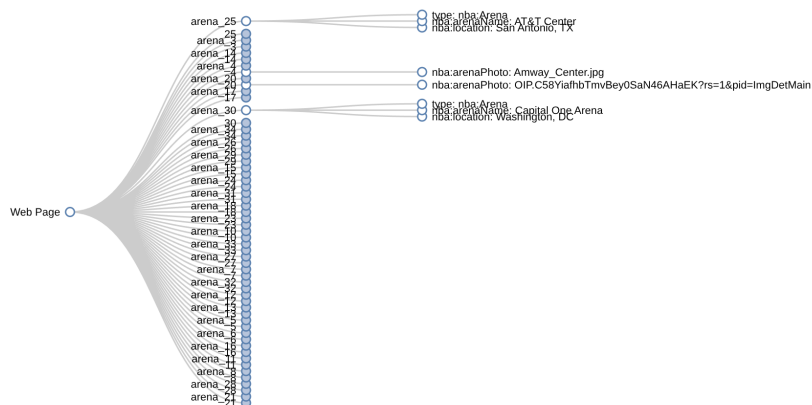


Figura 5.4: Resultado do parsing RDFa da página HTML das arenas

Apesar de ter sido possível incorporar RDFa em diversas páginas da aplicação, não conseguimos aplicar a anotação RDFa na página de listagem de jogadores (/players/). Isto deve-se ao facto de essa secção ser totalmente gerada dinamicamente via JavaScript, com os conteúdos dos jogadores a serem carregados e renderizados após o carregamento da página. Como o RDFa depende de atributos diretamente presentes no *HTML* estático, os dados renderizados via JavaScript não são reconhecidos por parsers RDFa como o RDFa Play, impossibilitando a extração automática dos triplos a partir dessa view.

5.2 Microformatos

Além do *RDFa*, os *Microformatos* foram de igual forma incorporados no *HTML* usando atributos de *classes* específicos, fazendo com que as anotações sejam legíveis por humanos e analisáveis por máquinas. Os microformatos mais utilizados incluem:

- **h-card**: para marcar dados de pessoas e organizações (por exemplo, jogadores, equipas, arenas, ...);
- **p-name**, **u-photo**, **p-locality**, **p-org**, **dt-start** e **dt-bday**: para anotar valores estruturados como nomes, imagens, locais e datas.

O Microformatos foram incorporados em:

- **/jogador/id/** - *templates/player.html*:

```
{
  "items": [
    {
      "type": [
        "h-card"
      ],
      "properties": {
        "name": [
          "A.C. Green"
        ],
        "org": [
          "Miami Heat"
        ],
        "country-name": [
          "United States"
        ],
        "note": [
          "Oregon"
        ],
        "photo": [
          {
            "value": "https://i.pinimg.com/474x/f4/94/12/f494128ef59aac8eb01ac0a12a64472b.jpg",
            "alt": "A.C. Green"
          }
        ]
      }
    }
  ]
}
```

Figura 5.5: Resultado do parsing dos Microformatos da página HTML do jogador

- **/coach/id/** - *templates/coach.html*:

```
{
  "items": [
    {
      "type": [
        "h-card"
      ],
      "properties": {
        "name": [
          "Anthony Grant"
        ],
        "photo": [
          {
            "value": "http://commons.wikimedia.org/wiki/Special:FilePath/Anthony%20Grant%20at%20Airport.JPG",
            "alt": "Anthony Grant"
          }
        ],
        "url": [
          "https://www.wikidata.org/wiki/Q4772615"
        ]
      },
      "lang": "pt"
    }
  ],
  "type": [
    "h-card"
  ]
}
```

Figura 5.6: Resultado do parsing dos Microformatos da página HTML do treinador

- **/season/id/** - *templates/season.html*:


```
{
  "items": [
    {
      "type": [
        "h-card"
      ],
      "properties": {
        "org": [
          "Baltimore Bullets"
        ],
        "url": [
          "/equipa/1610610024"
        ]
      },
      "lang": "pt"
    },
    {
      "type": [
        "h-card"
      ],
      "properties": {
        "org": [
          "Indianapolis Olympians"
        ],
        "url": [

```

Figura 5.7: Resultado do parsing dos Microformatos da página HTML da temporada

- */arenas/ - templates/arena.html*:

```
{
  "items": [
    {
      "type": [
        "h-card"
      ],
      "properties": {
        "name": [
          "AT&T Center"
        ],
        "locality": [
          "San Antonio, TX"
        ],
        "org": [
          "San Antonio Spurs"
        ],
        "photo": [
          {
            "value": "https://th.bing.com/th/1d/R.86c5fbd7b2f1479268f74bc68484975e7rik=270EYXP15ip2CQ&pid=Ingraw&w=0",
            "alt": "AT&T Center"
          }
        ]
      },
      "lang": "pt",
      "children": [

```

Figura 5.8: Resultado do parsing dos Microformatos da página HTML da arena

5.3 Validação

As anotações semânticas foram testadas usando ferramentas como:

- **RDFa Play**: para extrair e validar os dados RDFa;
- **Microformats Parser (pin13.net)**: para fazer parsing e verificar a conformidade com os microformatos.

6. Funcionalidades adicionais da UI

A interface foi significativamente melhorada e enriquecida com novas funcionalidades que tiram partido de inferência semântica e integração com fontes externas de dados. Estas melhorias contribuíram não só para uma experiência de utilizador mais completa, permitindo o acesso a informação inferida automaticamente mas também a dados externos adicionais para complementar os dados já existentes no dataset original.

6.1 Inferência na página dos detalhes do jogador

Na página de perfil de cada jogador foram adicionadas funcionalidades que mostram dados inferidos automaticamente a partir dos factos RDF e das regras SPIN:

- **Última equipa:** última equipa jogada pelo jogador com base na última participação na NBA do mesmo (`nba:lastTeam`);
- **Idade no ano do draft:** calculada subtraindo o ano do draft à data de nascimento (`nba:draftAge`);
- **Intervalo de carreira:** baseado nas épocas de participação do jogador em temporadas regulares (`nba:careerStart`, `nba:careerEnd`);
- **Jogadores que se estrearam (*drafted*) com ele:** lista de jogadores que participaram na mesma época e na mesma equipa na temporada de estreia.

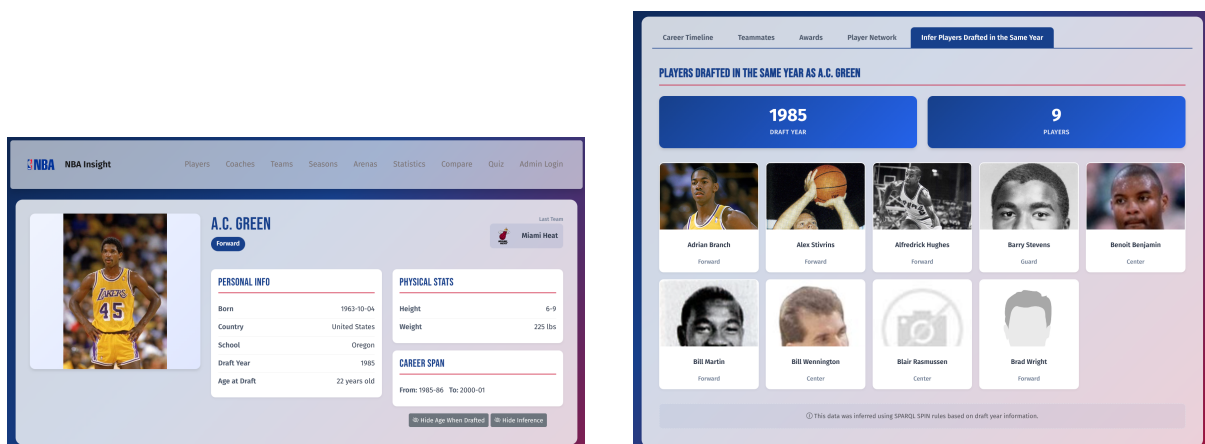


Figura 6.1: Novas funcionalidades da página do jogador

6.2 Inferência na página dos detalhes da equipa

A página de cada equipa também foi enriquecida com informação inferida com base em regras SPIN. Anteriormente, o campo **Conferência** era obtido indiretamente como atributo associado à divisão, como referido anteriormente. No entanto, passou a ser inferido e agora é propriedade direta da equipa. Na interface em si não houveram alterações:

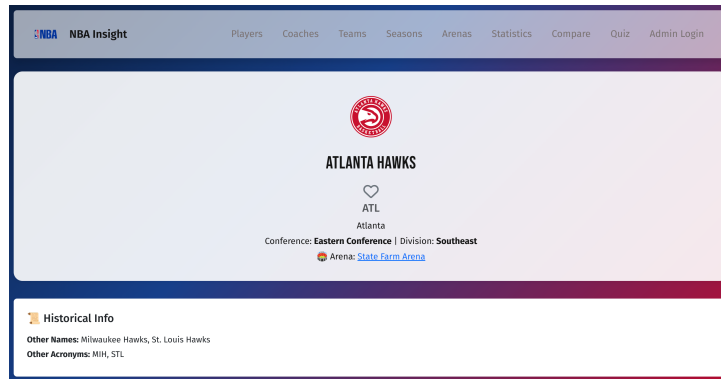


Figura 6.2: Propriedade "conference" na página dos detalhes da equipa

6.3 Integração com Wikidata — Prêmios do Jogador e Treinadores

A interface também foi expandida com dados obtidos externamente:

Prêmios de jogadores

Na tab *Awards* da página dos detalhes jogador, os prêmios individuais são obtidos dinamicamente a partir de queries SPARQL ao endpoint da *Wikidata*.

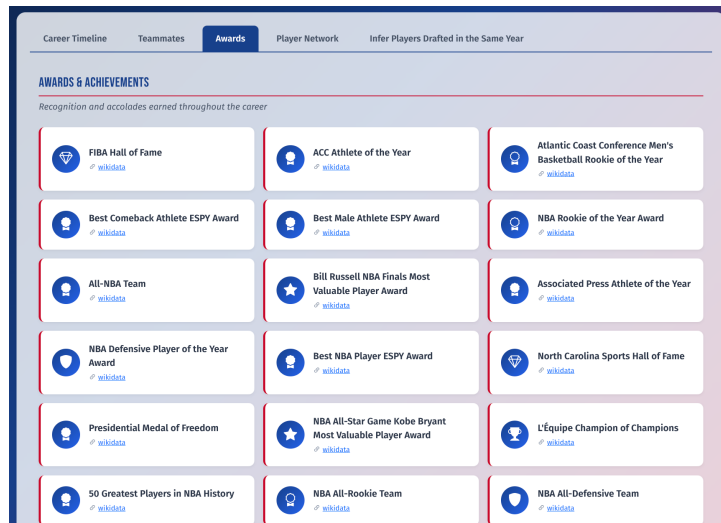


Figura 6.3: Nova tab dos premios do jogador

Listagem e detalhe de treinadores

Foram adicionadas duas novas páginas:

- *Coaches* (listagem dos treinadores), onde os dados dos treinadores são obtidos diretamente da *Wikidata* e apresentados em formato de cards;
- *Coach* (detalhes do treinador), os detalhes do treinador são obtidos, de igual modo, diretamente da *Wikidata*.

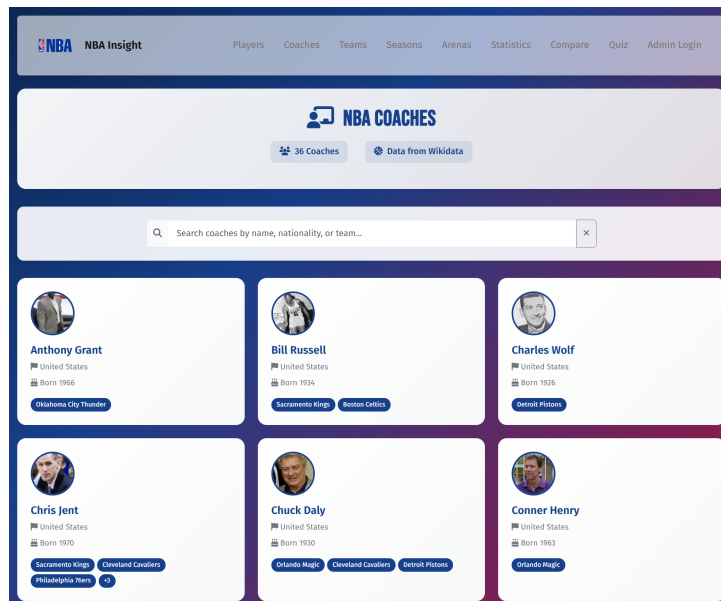


Figura 6.4: Nova página da listagem do treinadores

Ao clicar num treinador, é apresentada uma página de detalhes com:

- Informação pessoal (nascimento, género, nacionalidade, ...);
- Equipas treinadas com respetivos anos.

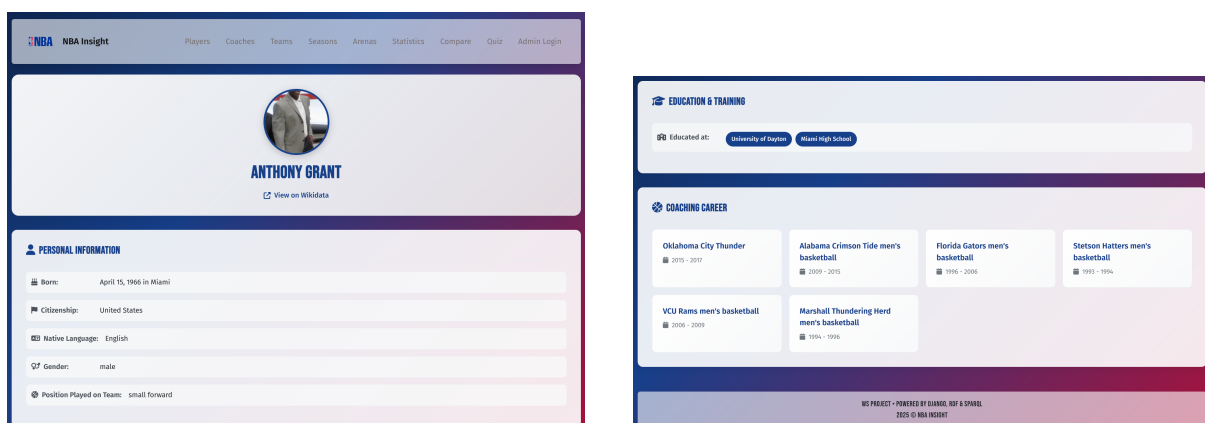


Figura 6.5: Nova página dos datalhes do treinador

Listagem de treinadores que passaram pela equipa

Na página dos detalhes da equipa foi adicionada um nova tab com a listagem dos treinadores que alguma vez passaram pela equipa.

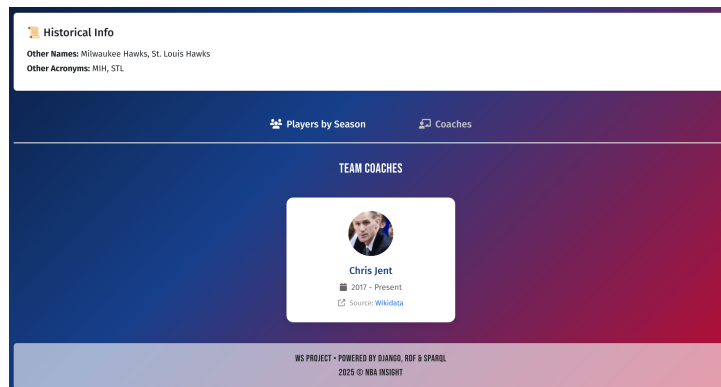


Figura 6.6: Nova tab com a lista dos treinadores na página dos detalhes da equipa

6.4 Integração com DBpedia — Arenas

Na página dos detalhes de cada arena foram integrados dados adicionais provenientes da *DBpedia*. Estes dados são carregados ao aceder à tab *Additional Details* e incluem:

- Custo de construção;
- Data de início da obra e data de abertura oficial;
- Arquitetos envolvidos, com imagem e nome.

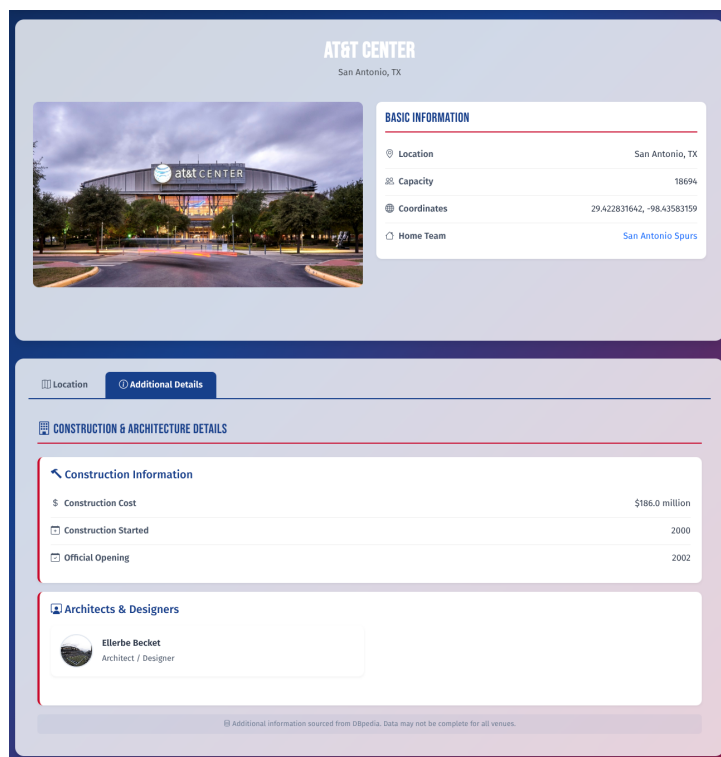


Figura 6.7: Nova tab dos detalhes de construção de uma arena

7. Conclusões

Este segundo trabalho prático permitiu consolidar e expandir ainda mais o sistema desenvolvido no TP1, demonstrando assim o poder que o valor que a Web Semântica trás na construção de aplicações ricas, interativas e semanticamente informadas.

A modelação dos nossos dados num Ontologia formal com RDFS e OWL permitiu estruturar e validar o domínio da NBA com coerência e expressividade. As regras SPIN implementadas tornaram possível inferir conhecimento não explícito e integrá-lo automaticamente na UI. Para além disto, a adição de novos dados vindos de fontes externas como *Wikidata* e *DBpedia* trouxe valor adicional ao sistema, possibilitando a apresentação de novos dados, que não existiam no dataset original. Na interface, estas melhorias refletiram-se em novas secções, visualizações intuitivas e dados inferidos visíveis para o utilizador. Adicionalmente, o uso de RDFa e microformatos permitiu publicar os dados de forma semântica, promovendo a interoperabilidade.

No geral, o projeto demonstrou o potencial da Web Semântica para criar aplicações mais inteligentes, informativas e interligadas.

8. Configuração para executar a aplicação

Repositório do Projeto

O código-fonte encontra-se disponível publicamente no GitHub em:

`https://github.com/RobertoCastro391/WS\_Projects/tree/main/NBAProject2`

Requisitos

- Ter o triplestore GraphDB instalado, e a correr, e criar um ambiente virtual *Python*, onde instalar as dependências que estão no ficheiro `requirements.txt`;
- Porta 7200 (GraphDB) e 8000 (Django) livres no sistema;

8.1 Executar a aplicação

Passos

1. Clonar o repositório e aceder à pasta do projeto:

```
git clone https://github.com/RobertoCastro391/WS_Projects.git
cd WS_Projects/NBAProject2
```

2. Criar e ativar um ambiente virtual:

```
python3 -m venv venv
source venv/bin/activate      # Em Linux/macOS
venv\Scripts\activate        # Em Windows
```

3. Instalar as dependências:

```
pip install -r requirements.txt
```

4. Iniciar o servidor Django:

```
python manage.py runserver
```

Configuração Automática do GraphDB

Ao rodar o comando para iniciar o server Django, o ficheiro `startup.py` é chamado automaticamente. Este ficheiro:

- Aguarda até o GraphDB estar acessível;
- Verifica se o repositório `NBA_G4` já existe;
- Se não existir, cria o repositório com base no ficheiro `data/nba-config.ttl`;

- Carrega automaticamente os factos e a ontologia dos ficheiros `data/facts.n3` e `data/nba_ontology.n3`, respetivamente, para o GraphDB.

Este processo é iniciado via `apps.py` ao arranque da aplicação.

O utilizador pode aceder:

- à aplicação Django via: `http://localhost:8000`
- ao GraphDB via: `http://localhost:7200`

8.2 Ficheiros Importantes

- `data/nba-config.ttl`: configura o repositório no GraphDB;
- `data/facts.n3`: triplos com factos puros, sem a ontologia;
- `data/nba_ontology.n3`: ficheiro contendo apenas a ontologia;
- `data/nba_ontology_facts.n3`: ficheiro combinado com ontologia e factos, usado para testes no Protégé;
- `data/spin_rules.py`: módulo Python que aplica as regras SPIN programaticamente, usando SPARQLWrapper;
- `app/data_complementation.py`: módulo de integração com fontes externas (Wikidata e DBpedia) que implementa consultas SPARQL para enriquecer os dados locais com informações adicionais;
- `app/startup.py`: script de configuração automática do repositório e carregamento de dados, invocado ao iniciar o Django;
- `requirements.txt`: lista de dependências Python necessárias ao projeto;
- `manage.py`: ficheiro principal do Django que permite iniciar a aplicação.